

Mathematical modeling of personalized learning and learning trajectories

Introduction

The modern education system cannot be imagined without infrastructure, many of whose components are prefixed with "electronic" or "digital" (e-learning, digital pedagogy, e-lectures, etc.). Computer-based learning support continues to evolve alongside the advancement of information technology. Support for constructivism and e-learning, as well as an approach to didactic engineering, is discussed in [1, 2]. Rozitsa Doneva and her colleagues [3] compare traditional and e-learning based on key learning elements. They describe the advantages and disadvantages of several learning platforms (Moodle, LAMS, PeU) and present the key functional capabilities of the BEST system, which incorporates all the positive aspects of the previous ones.

The modern approach to learning conceptualizes learning as knowledge construction and incorporates the concept of formal knowledge representation, which has long existed as a major branch of artificial intelligence [4, 5]. One of the rapidly developing areas of knowledge representation (modeling) in computers is semantic networks [6, 7]. Like other information technologies, semantic networks are also used in education. A study by Christanson and his colleagues [8] presents a solution to the problem of detecting knowledge gaps in textbooks using 10 linear algebra textbooks as an example, based on semantic networks.

Downes [9] explains learning objects with metadata to support reusability and accessibility for software learning agents. He also advocates an approach in which e-learning course creation, like software development, should be designed with:

- 1) RAD concepts, where a product is developed using reusable components. An online course is viewed as a system constructed from reusable learning objects;
- 2) concepts of object-oriented programming, where learning objects will be created on the basis of a certain class in a class hierarchy and can be in some relationship (association, aggregation, ...) with other objects - such as a test object, a course, etc.;
- 3) open standards concepts (LOM, SCORM) to enable distributed learning environments and learning objects developed by different authors in different languages, platforms and located on different computers to work together [16].

The discussion is primarily about course creation; semantic information (metadata) is used to integrate learning objects into the e-course and to describe the hierarchical structure. The formalism for describing metadata—as in IMS and SCORM [13]—is XML, XSL, and CSS.

The article by Arnold Kiva and his colleagues [14] describes a semantic learning network, an algorithm for constructing it, and its visualization. An algorithm for determining the logical sequence between the main concepts of a subject is presented. Specifically, semantic networks for the subjects "Economic Cybernetics," "Algorithms and Programming," and "Computational Mathematics" are considered. The topological characteristics of the network are discussed, based on properties such as vertex degree, centrality, clustering, etc.

The modern global network (Web3) includes a semantic component and is called the Semantic Web [15]. Many educational systems have been developed based on this type of network and are called the Semantic Learning Web. The basic principles of operation of such systems are described in the works of Stutt, Motta, and Devedzik [10, 11].

Standards were developed to integrate a course and a learning platform developed by a teacher and programmer who were not familiar with each other, to ensure course interactivity, and to obtain the necessary feedback. The SCORM standard describes the relevant mechanisms in detail [13]; these have been refined as technology has evolved [18].

This article is an attempt at mathematical modeling of the process of personalized learning [19], which, in turn, is a component of constructivism [20].

Based on the model described below, it is possible to specify a software system for machine support of intelligent analysis of the learning process (trajectory) – to identify rules (associations), similarities, differences and anomalous phenomena in the didactics used and in the learning trajectories of students [21, 22].

When didactically processed educational material has the ability to be adjusted depending on the needs of the student to improve the quality (bringing the result of learning closer to the didactic goal) and the effectiveness of learning (in terms of time and other possible resources), then we speak of personally-oriented learning [19].

This article examines the learning process as an object of study. The didactic goal is conceptualized as knowledge (Objective — O). The learner's current level of mastery (memorization, understanding, application, analysis, synthesis, comparative evaluation, mastery, proficiency [5]) is also conceptualized as knowledge (Comprehension — C).

Knowledge (an internal representation in computer memory) has the form of a certain labeled graph, where the nodes correspond to certain facts or generalized concepts, and the arcs denote relationships or associations between the nodes (ontology, semantic network, frame, ...) [6]. Further in the text, when speaking about knowledge, if there is no ambiguity, we mean a certain graph corresponding to knowledge.

The quality of learning is represented as a certain value, semantically meaning the closeness, similarity or distance (Distance - D) between the level of mastery of the learner C and the learning goal O. For given two graphs C and O, the distance can be determined [21] based on:

- maximum distance of common subgraphs;
- similarities of structures;
- editing operations;
- nuclear functions.

Since we are talking about knowledge construction, the most suitable method is based on editing operations, the algorithms of which are described in detail by Aggarwal in Chapter 17 of his textbook [21].

1. Learning Object Repository

A learning object is an explanation of some content—an SCO [7]—of the learning material, along with the necessary metadata for interaction with the learning platform. An SCO can be an elementary asset (text, procedure, image, audio, video, etc.) or some

combination of elementary assets and SCOs. Downs [9] calls such objects learning objects.

A learning object repository (R) is a repository containing a set of learning objects structured by metadata. Each learning object in the repository is described by a triple of the form $\langle \text{Pre}, \text{Exp}, \text{Res} \rangle$, where Pre is the prior knowledge necessary to understand the explanation, Exp is the explanation itself (the learning content), and Res is the knowledge the learner acquires after studying the object. The repository serves as the basis for the automatic construction of didactics: the algorithm for finding a suitable learning object is based on matching the required prior knowledge and the object's outcome with the specified didactic goal.

2. Didactics

We model the concepts of didactics for teaching some knowledge.

Definition (Didactics): The didactics of teaching (hereinafter simply didactics) is a triple $D = \langle \text{Pre}, \text{Res}, G \rangle$, where Pre is some preliminary knowledge necessary for teaching a given knowledge; Res is the didactic learning goal—the knowledge that the learner obtains as a result of learning based on a given didactics; $G = \langle N, E \rangle$ is the didactic graph—a directed graph of didactically processed content. N is the set of graph vertices; E is the set of graph arcs.

The vertices of the didactic graph G correspond to events in the educational process, that is, triplets of the form $\langle \text{Pre}, \text{Exp}, \text{Res} \rangle$ are assigned to the vertices, where:

Exp (explanation) — object of study;

Pre (Precondition) - some preliminary knowledge required to understand Exp, that is, the explanation is based on this knowledge;

Res (Result) — postcondition, result — some knowledge that the student will (should) acquire as a result of explanation Exp.

The semantics of the vertex means that "if the learner knows Pre, then after explaining Exp he (she) will know Res."

The arcs of the didactic graph G correspond to "follow-up" relationships among the graph's vertices. If there is an arc from vertex n_i to vertex n_j , this means that explanation n_j .Exp follows (is based on) explanation n_i .Exp.

For node n , $n.\text{Pre} \subseteq n.\text{Res}$ is always true, and $\Delta n = n.\text{Res} - n.\text{Pre}$ is the knowledge that the learner receives after explaining $n.\text{Exp}$, and is called the didactic effect of node n .

We assume that for the $\forall$ vertex n of didactics the following conditions are met:

- 4) $\forall j \in \text{predecessor}(n), \neg(n.\text{Pre} \subseteq (\bigcup_{i \in \text{predecessor}(n)} i.\text{Res} - \Delta j))$ — the condition of necessity of any predecessor;
- 5) $n.\text{Pre} \subseteq \bigcup_{i \in \text{predecessor}(n)} i.\text{Res}$ — the sufficiency condition of predecessors;
- 6) $\bigcup_{i \in \text{predecessor}(n)} i.\text{Res} = \min\{\exists x \in \mathfrak{R}(N) : n.\text{Pre} \subseteq \bigcup_{i \in x} i.\text{Res}\}$ is the condition for minimality of predecessors.

Here: $\text{predecessor}(n)$ is the set of predecessors of vertex n ; $\mathfrak{R}(N)$ is the set of all subsets of set N .

Condition 1 states that node j is necessary for understanding (mastering) n .Exp. That is, without mastering the explanation contained in j 's predecessor, it is impossible to master its successor n . Condition 2 states that to master a successor, it is sufficient to master all of its predecessors. That is, the predecessors of n are sufficient for mastering explanation n . Condition 3 states that the set of nodes' predecessors constitutes the minimum amount of knowledge required to master the explanation of the successor. That is, any other set of nodes whose mastery is sufficient for mastering n contains more knowledge than the knowledge contained in all of n 's predecessors.

From the point of view of didactic construction, a node is suspended as a successor to a minimum number of nodes on which it logically depends; at the same time, the predecessors together constitute sufficient knowledge to understand the explanation of node n .

The initial vertex of a didactic is the one that has no predecessors. It may be the case that there are several initial vertices in a didactic— $n_1, n_2, \dots, n_k$. Then they can be reduced to a single initial vertex— n_s , by connecting $n_1, n_2, \dots, n_k$ as successors of n_s , and setting $n_s.Res = \bigcup_{i \in \{n_1, \dots, n_k\}} j.Pre$ with an empty didactic effect ($\Delta n_s = \emptyset$), that is, $n_s.Pre = n_s.Res$. Further, without loss of generality, we assume that the didactic graph G has a single initial vertex n_s (denoted as Gn_s) with the precondition of the didactic $G.Pre$. We call it the precondition of the didactic $D.Pre$.

A terminal vertex of a didactic is one that has no successors. Similarly, a didactic may have several terminal vertices $n_1, \dots, n_k$. They can also be reduced to a single one, n_f , by connecting $n_1, \dots, n_k$ as predecessors of n_f and setting $n_f.Pre = \bigcup_{i \in \{n_1, \dots, n_k\}} j.Res$ with an empty didactic effect ($\Delta n_f = \emptyset$), that is, $n_f.Res = n_f.Pre$. Further, without loss of generality, we assume that a didactic has a single terminal vertex n_f (denoted as Gn_f) with a didactic result Res . We denote the result of a didactic as $D.Res$.

Figure 1 shows a didactic graph where four initial vertices (a, b, c, d) are reduced to one (n_s), and two final vertices (j, k) are reduced to one (n_f).

3. Design of didactics

Let's assume that the R repository contains a sufficient number of training objects for a given learning goal. Consider the problem of adding a new node.

3.1. Determining the Predecessors of a Vertex

Determining the predecessors of a node consists of two algorithms: FindPredecessors and CreatePredecessors. CreatePredecessors generates a list of all potential predecessors (PotentialList) and then calls FindPredecessors to determine the optimal list of predecessors (OptimalList).

FindPredecessors determines a list of predecessors in which the didactic effects do not overlap, meaning that no two nodes share the same knowledge. This is achieved by using the Δ hor variable, which accumulates the didactic effects of the detected predecessors. Furthermore, the total knowledge in OptimalList will be minimally covering, meaning that it will not contain any extra knowledge to master the explanation of node n . CreatePredecessors then creates predecessor nodes for each member of OptimalList and (if any) an additional node for the remaining missing knowledge $Pre - MinRes$, required to master the explanation of node n .

Didactic graph: Model of the learning trajectory

Reconstruction of the structure and logic of a didactic graph (DAG) for visualizing the learning process as a sequence of logically related educational elements.

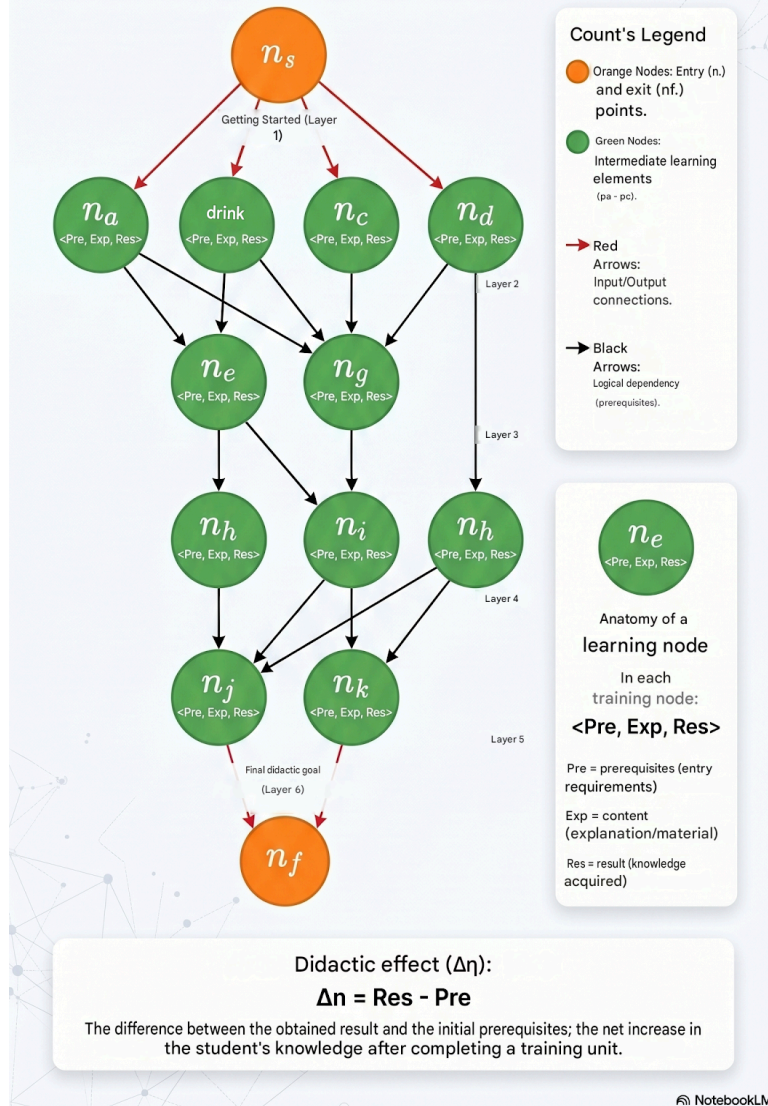


Fig. 1. Didactic graph.

Global variables used in algorithms:

- List of all potential predecessors - PotentialList;
- Required general knowledge for precursors - Pre;
- Minimum Resume of Predecessor Results - MinRes;
- Optimal list of predecessors - OptimalList.

Alg. FindPredecessors (n, Res, Δ_{hor} , MinRes, CurPred)

Input: Current node in PotentialList is n, current union of predecessor results is Res, current union of predecessor effects is Δ_{hor} , current predecessor list is CurPred.

Output: Optimal list of predecessors - OptimalList.

Method:

```
m = Next(PotentialList, n);
If  $\Delta_{hor} \cap \Delta_n = \emptyset$  then           // Condition of horizontal non-intersection Then
    y = Res  $\cup$  n.Res; Incl(CurPred, n);
    If  $y \supseteq \text{Pre}$  then                 // The number of predecessors is sufficient Then
        If  $y \subset \text{MinRes}$  then         // Adjust the optimal list Then
            MinRes = y; OptimalList = CurPred;
        End
    Else
        If m  $\neq$  null Then
            FindPredecessors (m, y,  $\Delta_{hor} \cup \Delta_n$ , CurPred); // Taking into
            account ver. N
        Else
            If S =  $\emptyset$  Then MinRes = y; OptimalList = CurPred; End
        End
    End
End
Excl(CurPred, n);
End
If m  $\neq$  null Then
    FindPredecessors(m, Res,  $\Delta_{hor}$ , CurPred); // Without taking into account ver. n
End
```

It's important that the precondition (n.Pre) doesn't overlap with the explanation, meaning that the explanation doesn't directly or indirectly build on itself. To achieve this, CreatePredecessors uses the Δ_{ver} variable, which accumulates didactic effects along the precedence path.

Alg. CreatePredecessors(n, D, R, Δ_{ver})

Input: The vertex for which predecessors are determined — n ; the current (constructed so far) didactic graph — D ; the repository of elementary educational contents — R ; the union of didactic effects along the path of precedence — Δ_{ver} .

Output: Set S of all predecessors of node n and missing knowledge m .

Method:

Pre = $n.Pre$; OptimalList = $\emptyset$; PotentialList = $\emptyset$;

For $\forall d \in B$ (first by D , then by $R-D$) Do

If $(\Delta d \cap \Delta_{ver} = \emptyset) \ \& \ (\Delta d \cap Pre \neq \emptyset)$ Then $d \rightarrow PotentialList$ End

All

FindPredecessors(PotentialList.first, $\emptyset, \emptyset, PotentialList, \emptyset$);

For $\forall k \in OptimalList$ Do

GenPred(n, k);

CreatePredecessors($k, D, R, \Delta_{ver} \cup \Delta k$)

End

If $MinRes \subset Pre$ Then

$n_s.Res = n_s.Res \cup Pre - MinRes$;

GenPred(n, n_s);

End

Additional functions used:

- Incl(List, n) — includes element n in the List;
- Excl(List, n) — excludes element n from the List;
- Next(List, n) — returns the next element after element n from the List;
- GenPred(n, k) - creates an arc (k, n) ;
- CreateNode(Pre, Exp, Res) — creates a node with the triple $\langle Pre, Exp, Res \rangle$;
- Predecessors(n) — a list of all predecessors of vertex n .

3.2. Generation of didactics

Didactic generation is based on the learning object. The DidacticGen algorithm creates a start and end node with an empty didactic effect and generates all the predecessors of the end node by calling CreatePredecessors.

Alg. DidacticGen (O, R)

Input: Repository of elementary educational contents - triplets $\langle \text{Pre}, \text{Exp}, \text{Res} \rangle$ - R and didactic goal (Objective) - O.

Output: Didactics D, where $D.\text{Res} \supseteq O$.

Method:

$n_f = \text{CreateNode}(O, \emptyset, O)$; $D = n_f$;

$n_s = \text{CreateNode}(\emptyset, \emptyset, \emptyset)$;

$\text{CreatePredecessors}(n_f, D, R, \emptyset)$;

Statement 1. The didactics constructed by the above algorithm is acyclic.

Proof. Suppose there exists a cycle. Then we can imagine a sequence of nodes $e_{i1}, e_{i2}, \dots, e_{ik}$, forming a path, where $e_{i1} = e_{ik}$. Then, since in the CreatePredecessors algorithm the variable Δ_{ver} accumulates the didactic effects along the path and does not allow the didactic effect of the next node along the path to intersect with Δ_{ver} , it turns out that the didactic effect of a node is added twice. This contradiction proves the given statement. ■

Sub-didactics: Didactics D_1 is a sub-didactics of D_2 if $D_1.O \subseteq D_2.O$ and $D_1 \subseteq D_2$ in the graph sense, that is, $N_{D_1} \subseteq N_{D_2} \ \& \ E_{D_1} \subseteq E_{D_2}$.

Equivalent didactics: Didactics D_1 and D_2 are equivalent if $O_{D_1} = O_{D_2} \ \& \ D_1.\text{Pre} = D_2.\text{Pre}$.

Detailed node: For a node n of a didactic D_1 , a didactic D_2 is a detailing if n and D_2 are equivalent.

Detailed didactics: For given didactics D_1 and D_2 , we say that D_2 details D_1 if there is at least one vertex in D_1 that D_2 details, and otherwise they coincide.

4. Training

Currently, generating didactics is a time-consuming and cumbersome task for conventional servers. Therefore, once generated, didactics are typically not deleted after use, but stored in a repository for future reuse. Therefore, when searching for a learning element in the repository, there is a slight chance of finding a pre-generated didactics.

From time to time, it is necessary to restructure the stored teaching methods to incorporate newly added learning elements and teaching methods. However, this issue is beyond the scope of this article.

When searching the repository and discovering a didactic that meets the stated goal, there's no need to build it from scratch—we can use the existing one. Next, we model the student's learning process based on the pre-designed didactic.

The learner does not necessarily have to study all the learning materials assigned to the nodes of the didactic if the learning goal is a subset of the goal of the constructed didactic. They may not start from the beginning, as they are already familiar with some of the

material, or they may not study it completely, meaning they have a learning goal that is a subgoal of the goal of the given didactic. Thus, it will be necessary to "cover" not the entire didactic, but some sub-didactic. The learner sequentially studies the explanations of all nodes of the sub-didactic in a certain order. Based on condition 2 of the didactic, the learner will master the learning material by first studying all the predecessors of a node, and then the successor node itself. The order in which the predecessors are studied is not important.

Definition (Training): We call training such a linearization (numbering) of the vertices of sub-didactics, in which the above condition is not violated.

Let's assume a learner (L) has a goal—achieving some knowledge (LO)—with prior preparation (awareness) LC. We assume that $LC \subset LO$, otherwise there is no point in learning. To utilize a constructed didactic D, it is necessary that $(D.Pre \subseteq LC) \& (DO \supseteq LO)$, that is, the learner can master the educational material of the didactic and, after mastering it, achieve their goal. We will call such a didactic suitable for learner L.

4.1. Determining the minimum goal that satisfies the learner

An algorithm for determining the minimum goal satisfying the learner, based on parameters O—the learning object and D—the didactics—determines a vertex n of the didactics D such that $n = \text{Min}(k.Res : \forall k \in DN) \& O \subset n.Pre$. The algorithm scans all vertices of the graph D, starting with Dn_f , from bottom to top through all predecessors whose result covers O.

Alg. MinObjective (O, D, m)

Input: Objective (LO) and didactics D, such that $LO \subset D.Res$.

Output: A vertex minObjective $\in DN$ such that $LO \subseteq \text{minObjective}.Res$, and for $\forall n \in DN$, where $\text{minObjective} \neq n$: $\text{minObjective}.Res \subseteq n.Res$ or $n.Res \subset LO$. That is, minObjective is a minimal vertex covering LO. The minimal subdidactic $D_{\min}$ that ensures LO

Method: Let Q be some queue.

$m = Dn_f$; Incl(Q, Dn_f);

While $Q \neq \emptyset$ Do

$n = \text{Excl}(Q, n)$;

If $n.Res \subset \text{minObjective}.Res$ Then $\text{minObjective} = n$; End

For $\forall k \in \text{Predecessors}(n)$ Do

If $k \notin Q$ Then Incl(Q, k); End

End

End

Note that we should not terminate the search through the predecessors of a node if Res of the node does not cover LO, because Res includes the knowledge of $Pre + \Delta$, and Pre is a subset of Res of its predecessors; thus, among the predecessors there may exist one such that its Res covers LO.

If for given L and D , $\text{MinObj}(LO, D) = Dn_s$, then this means that the given didactics is too complex for the given student and preliminary preparation is required to study it.

5. Learning tree

The student's learning progresses through the didactics to a peak with a minimum goal that satisfies the student (covering the student's goal, $\text{Res} \subseteq LO$), and begins with peaks that lead to the minimum goal and are simple enough for the student to master (covered by the student's awareness, $\text{Pre} \subseteq LC$). After determining the minimum learning goal, the minimum sub-didactics necessary for learning are determined. It is this minimum sub-didactics that the student must "pass" to achieve the goal LO .

The LearnTree learning tree definition algorithm defines a tree that serves to achieve the learning goal of the learner.

Alg. LearnTree ($LC, D, \text{minObjective}$)

Input: Preliminary preparation of LC , didactics D with the minObjective vertex.

Output: A subgraph formed from the marked vertices of the didactics.

Method: Let Q be some queue.

Incl($Q, \text{minObjective}$);

While $Q \neq \emptyset$ Do

$n = \text{Excl}(Q, n)$; Mark n ;

For $\forall k \in \text{Predecessors}(n)$ Do

If ($LC \subset k.\text{Pre}$) Then

If $k \notin Q$ Then Incl(Q, k); End

Else Mark n ;

End

End

End

PostOrder(minObjective);

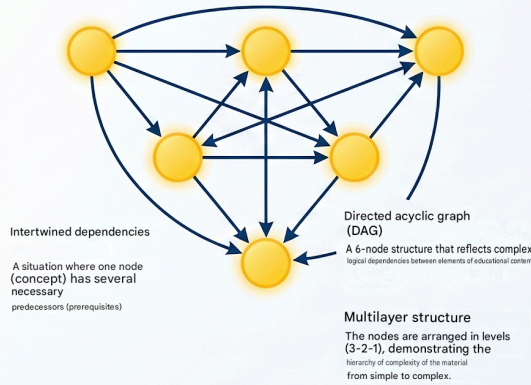
Here, PostOrder(n) is an ascending (left-hand or right-hand) numbering [24] of an acyclic graph, starting with vertex n . PostOrder numbers vertices like a tree, with one difference: when it encounters an already numbered vertex, it backtracks. If we remove the arcs connecting such vertices, we obtain a tree. We call such a tree LearnTree($LC, D, \text{minObjective}$) – a learning tree.

For example, if we number the following acyclic graph using PostOrder, we obtain a tree.

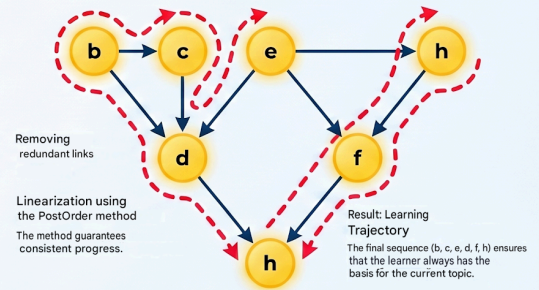
Building a Learning Tree: From Graph to Trajectory

Tree linearization using the PostOrder method (bottom-up traversal) allows for the formation of a sequential learning path, where knowledge is accumulated iteratively.

Part a) Acyclic graph of didactics



Part b) Learning Tree



PostOrder stage	Action in node	Effect
Ascent (Leaves)	Development of the base	L.C knowledge corresponds to the node's Precondition
Iteration	Transition to successor	Accumulation of didactic effect (D)
tree root	Achieving the goal	The resulting knowledge corresponds to Objective (L.O)

NotebookLM

Fig. 2. a) acyclic graph, b) tree obtained by PostOrder numbering.

Statement 2. Linearization of a learning tree by PostOrder is learning.

Proof. Since PostOrder is ascending, the successor's number will always be greater than any predecessor's. When a node in an acyclic graph branches, its predecessors are numbered first, followed by the node itself—the first time it is accessed from its successor. Subsequent accesses to other successors ignore the node. Thus, any successor has a greater number, meaning that PostOrder is a linearization that satisfies the learning condition. This proves the statement.

If a learner studies the materials assigned to the nodes of a learning tree by traversing all the nodes of the tree, having previously numbered them using PostOrder, then for any node n of the tree that the learner begins studying, $LC \supseteq n.Pre$ holds. This means that the learner does not end up in a situation where the current explanation is based on material they have not yet studied.

If we look at the learning tree of a given didactics — $LearnTree(D.Pre, D, MinObj(DO, D))$, then, by linearizing (sequentially linking) the educational materials assigned to the vertices of the learning tree according to PostOrder, we obtain a didactically revised **electronic textbook**.

Thus, the learner achieves their goal by studying the materials assigned to the nodes of the learning tree, traversing all the tree nodes according to PostOrder, starting with its leaves. The learner's knowledge accumulates iteratively, node by node: each time they traverse node n , the learner gains an increment of Δn . This model can be said to correspond to iterative (flexible) learning [26].

Literature

1. Clark RC, Mayer RE e-Learning and the Science of Instruction. Proven Guidelines for Consumers and Designers of Multimedia Learning. 3rd ed. Pfeiffer, 2011. 527 p.
2. Tchoshanov M. Engineering of Learning: Conceptualizing e-Didactics. UNESCO Institute for Information Technologies in Education, Moscow, 2013. 194 p.
3. Doneva R., Denev D., Totkov G. On the Didactic Principles, Models and E-Learning. International Journal "Information Theories and Applications".
4. Brachman RJ, Levesque HJ Knowledge Representation and Reasoning. Morgan Kaufmann, 2004. 381 p.
5. Bloom BS, Engelhart MD, Furst EJ, Hill WH, Krathwohl DR Taxonomy of Educational Objectives: The Classification of Educational Goals. Vol. Handbook I: Cognitive Domain. New York: David McKay Company, 1956.
6. Sowa JF (ed.) Principles of Semantic Networks: Explorations in the Representation of Knowledge. Morgan Kaufmann Publishers, 1991. 576 p.
7. Lehmann F. Semantic Networks. Computers Math. Applic. Vol. 23, No. 2–5, pp. 1–50, 1992.
8. Christianson NH, Blevins AS, Bassett DS Architecture and evolution of semantic networks in mathematics texts. Proc. R. Soc. A 476: 20190741. <http://dx.doi.org/10.1098/rspa.2019.074>
9. Downes S. Learning Objects: Resources For Distance Education Worldwide. International Review of Research in Open and Distance Learning. Vol. 2, No. 1, 2001.
10. Stutt A., Motta E. Semantic Learning Webs. Journal of Interactive Media in Education, Special Issue on the Educational Semantic Web, No. 10 (2004).
11. Devedzic V. Education and the Semantic Web. International Journal of Artificial Intelligence in Education, 14, 39–66 (2004).
12. IEEE LTSC: IEEE Learning Technology Standards Committee Glossary. IEEE P1484.3 Glossary Working Group, draft standard (2001).
13. ADL Initiative. Sharable Content Object Reference Model (SCORM). Advanced Distributed Learning Initiative, 2004. Available online at <http://www.adlnet.org/>
14. Kiv A., Soloviev V., Tarasova E., Koycheva T., Kolesnykova K. Semantic knowledge networks in education. E3S Web of Conferences 166, 10022 (2020) ICSF 2020. <https://doi.org/10.1051/e3sconf/202016610022>
15. Berners-Lee T., Hendler J., Lassila O. The Semantic Web. Scientific American, May 2001.
16. Hodgins W. Out of the past and into the future: Standards for technology enhanced learning. In: Ehlers U., Pawlowski J. (eds.). Handbook on Quality and Standardization in E-Learning, pp. 309–327. Springer Berlin, 2006.
17. Tane J., Schmitz C., Stumme G., Staab S., Studer R. The Courseware Watchdog: an Ontology-based tool for finding and organizing learning material. In: Fachtagung Mobiles Lernen und Forschen, Kassel, 2003.

18. Advanced Distributed Learning. SCORM 2004 4th Edition Overview. ADL Initiative, 2009.
19. Weimer M. Learner-Centered Teaching: Five Key Changes to Practice. 2nd ed. San Francisco: Jossey-Bass, 2013. 288 p.
20. Papert S., Harel I. Constructionism. Ablex Publishing Corporation, 1991.
21. Aggarwal CC Data Mining: The Textbook. Springer, 2015. 746 p.
22. Tan P., Steinbach M., Karpatne A., Kumar V. Introduction to Data Mining. 2nd ed. New York: Pearson Education, 2019. 1459 p.
23. Bloom BS et al. — see [5].
24. Aho AV, Hopcroft JE, Ullman JD Data Structures and Algorithms. Addison-Wesley, 1983.
25. Piaget J. The Equilibration of Cognitive Structures: The Central Problem of Intellectual Development. University of Chicago Press, 1985.
26. Sharples M. et al. Agile Methods for Technology-Enhanced Learning. In: Proceedings of the European Conference on Technology Enhanced Learning, 2014.